

Universidad Veracruzana

Región Poza Rica - Tuxpan

Facultad de Ingeniería en Electrónica y Comunicaciones

Ingeniería en Tecnologías Computacionales

Emulación del Procesador MIPS en lenguaje Java

Experiencia educativa: Microprocesadores y Microcontroladores

Presenta (n):

S24021393 Valdez Avila Moises Alejandro

S24006912 Velazquez Garcia Alexis David

S24006869 Del Angel Garcia Ian Yazid

S24006890 Perez Nuñez Genaro

10 de Octubre de 2026

"Lis de Veracruz: Arte, Ciencia, Luz"

Introducción

Desarrollar un emulador funcional de un microprocesador MIPS de 32 bits utilizando Java.

Y replicar fielmente el comportamiento del hardware y el ciclo de cómputo sin abstracciones de alto nivel.



MIPS

Origen en la Universidad de Stanford

Desarrollada en 1981 por el Dr. John Hennessy

MIPS significa

Microprocessor without **I**nterlocked **P**ipeline **S**tages

Microprocesador sin etapas segmentadas entrelazadas



Impacto Industrial de MIPS

- Adopción masiva en sistemas embebidos
- Presente en telecomunicaciones: routers Cisco
- En el entretenimiento: PlayStation 1 y 2, Nintendo 64
- Una arquitectura RISC que impulsó productos y mercados

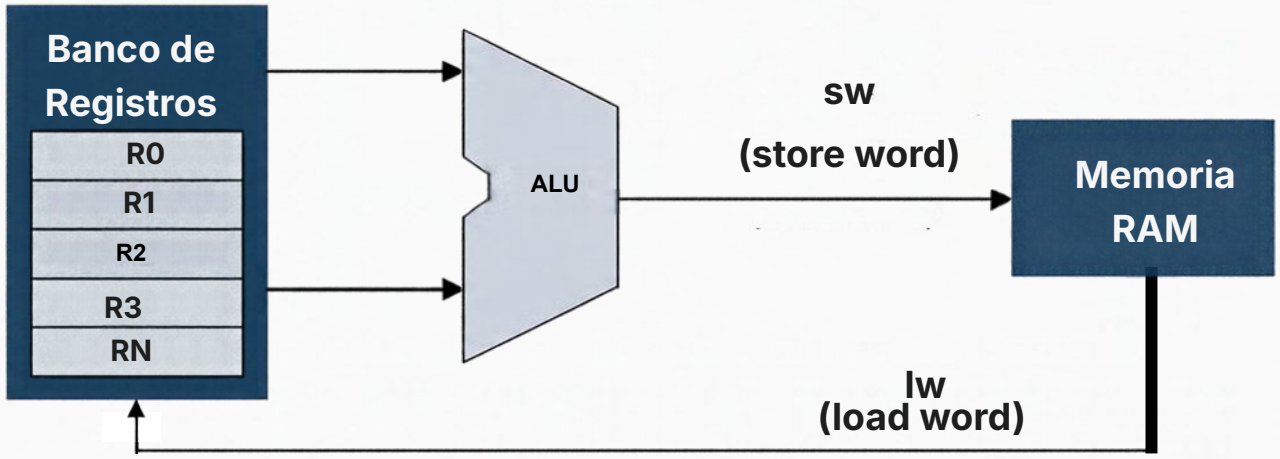


Filosofía Técnica y Arquitectura de MIPS

Diseño RISC con instrucciones de 32 bits de longitud fija

Los cálculos se realizan en registros

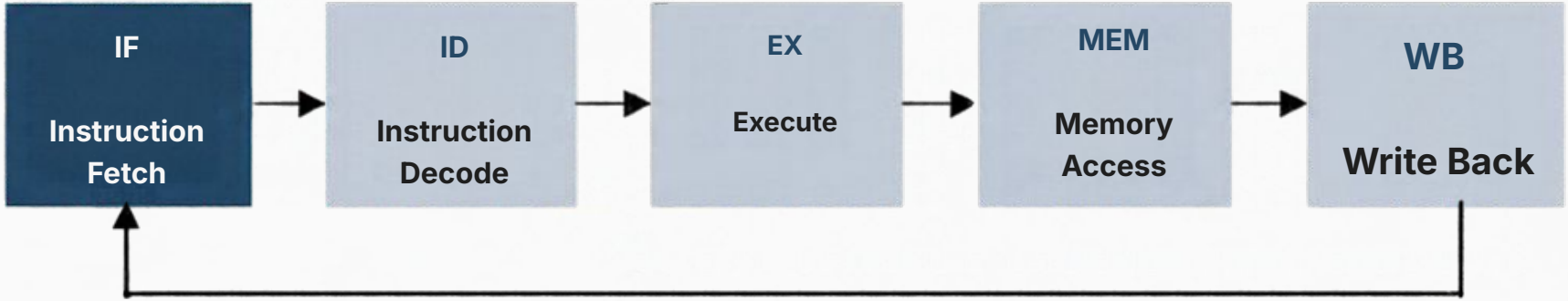
La memoria solo se accede con lw (load word) y sw (store word)



Filosofía Técnica y Arquitectura de MIPS

Ciclo de instrucción dividido estrictamente en 5 etapas lógicas

- **IF (Instruction Fetch):** Búsqueda de la instrucción en la ROM.
- **ID (Instruction Decode):** Decodificación y lectura de operandos.
- **EX (Execute):** Procesamiento matemático o lógico en la ALU.
- **MEM (Memory Access):** Lectura/Escritura en la RAM (si aplica).
- **WB (Write Back):** Escritura del resultado final en el registro destino.



Paso 1. Arquitectura del sistema de almacenamiento de datos



Contador de programa

Definición

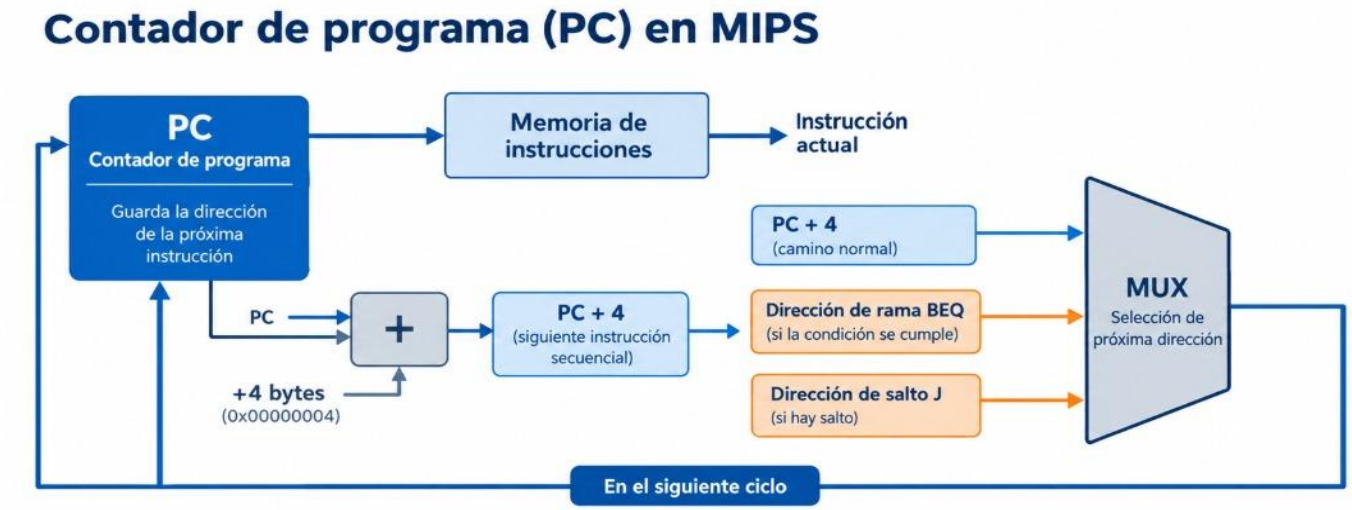
Registro especial (PC) que almacena la dirección de memoria exacta de la instrucción que la CPU está ejecutando actualmente.

Comportamiento físico en MIPS

Avanza automáticamente de 4 en 4 bytes tras cada fase de lectura (Fetch). Puede ser sobrescrito abruptamente por la ALU en caso de un salto condicional (beq) o incondicional (j).

Implementación en Java

Diseñado como una clase independiente (ProgramCounter.java) para aislar su circuitería.

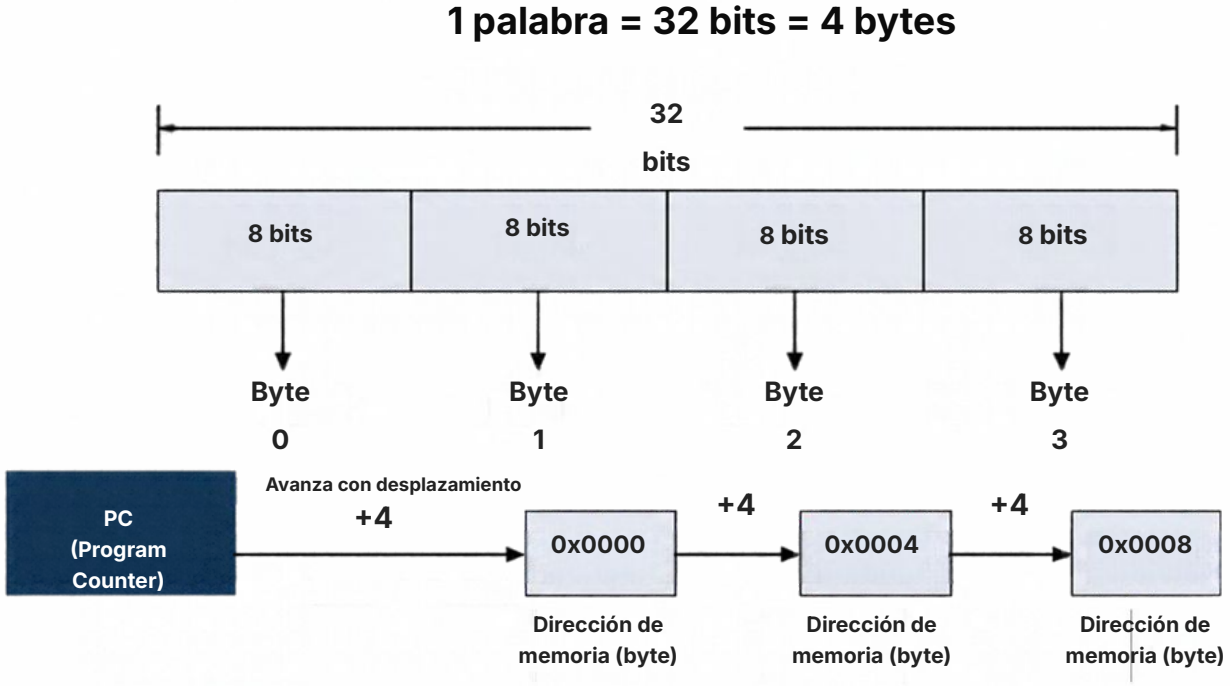


Ancho de palabra

- El ancho de palabra (Word) es la cantidad de bits que el procesador puede manejar, transmitir o procesar en una sola operación matemática o de memoria.

Estándar MIPS (32 bits)

- 1 palabra = 32 bits = 4 bytes
- **T**odo código máquina en MIPS mide exactamente una palabra (32 bits)
- Cada uno de los 32 registros generales tiene una capacidad máxima de 32 bits



Registros del procesador

Banco de memoria interna ultrarrápida

32
registros de propósito general
$zero a $ra

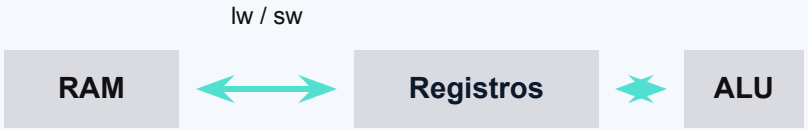
32 bits
1 palabra exacta
por registro

El área de trabajo de la ALU

En la arquitectura Load/Store, ninguna operación matemática se realiza directamente en la RAM.

$zero: siempre vale 0

El índice 0 está cableado físicamente a tierra (Hardwired to 0). Su valor no puede modificarse.



RegisterFile.java

Arreglo primitivo int[] de 32 posiciones.
La escritura ignora el índice 0: protección de hardware simulada.

```
public class RegisterFile {
    private final int[] registers
        = new int[32];

    public void write(int index, int value) {
        if (index != 0) {
            registers[index] = value;
        }
    }

    public int read(int index) {
        return registers[index];
    }
}
```

Implementación de la ROM

Firmware binario y lectura de instrucciones

String[]

ROM java encapsula un arreglo String[] que emula el firmware quemado en el chip.

32 caracteres

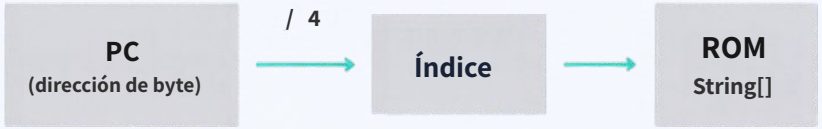
Cada instrucción se almacena como una cadena binaria de 32 caracteres.

Dirección a índice

El método read convierte la dirección de byte del PC
en índice dividiendo entre 4.

De binario a entero

La cadena binaria se convierte en tiempo real a un entero para que la Unidad de Control aplique máscaras de bits.

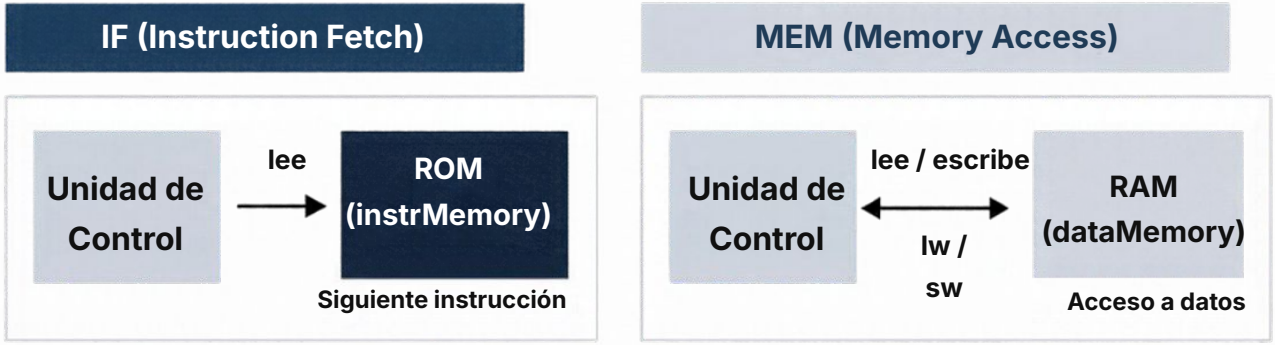



```
public class ROM {

// Firmware quemado en memoria (ej. Algoritmo de Fibonacci)
private final String[] firmwareBinario = {
   00100000000010000000000000010100
      }
  };


public int read(int address) {
    // Convierte la dirección de byte (PC) a índice de
arreglo
        int index = address / 4;
    // Parsea el String binario a un entero de 32 bits
    return (int) Long.parseLong(firmwareBinario[index], 2);
  }
}   }
```

Acceso simultáneo en el ciclo: la ventaja del modelo Harvard

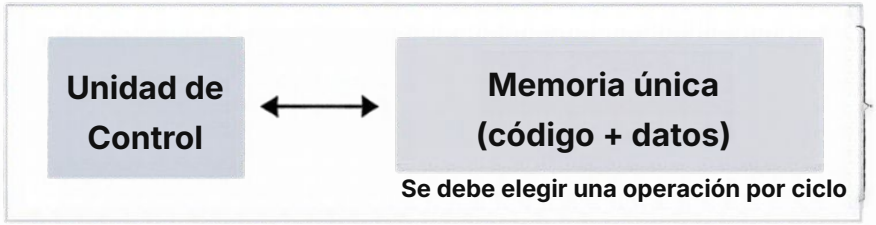


instrMemory y dataMemory son objetos Java **distintos.**

↓

El procesador puede acceder teóricamente a ambas memorias en el mismo ciclo **sin colisión.**

Comparación: modelo Von Neumann



- Leer instrucción (código)
- Leer/Escribir datos (lw / sw)

No ambas a la vez.

→ **Posibles pausas o esperas**

El modelo Harvard evita contención al separar físicamente código y datos.

Separación en el código: ROM vs. RAM

ROM.java

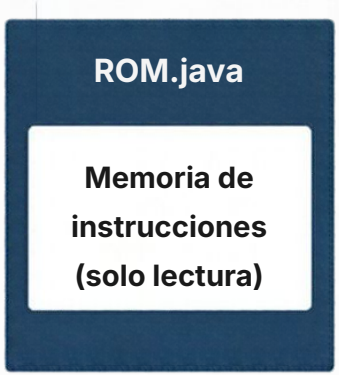
Memoria de instrucciones (solo lectura)

Almacena el código binario de los programas, como Suma o la Integral.

RAM.java

Memoria de datos (lectura y escritura)

Guarda variables en tiempo de ejecución, como los números de un arreglo para calcular un promedio y los resultados intermedios.





Prueba técnica: validación de almacenamiento

Integridad del modelo Harvard y persistencia del banco de registros

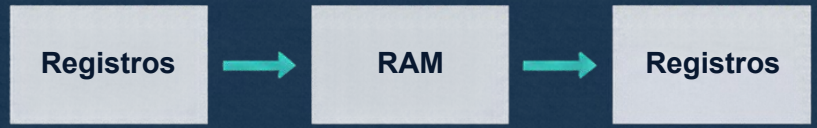
Objetivo

Validar el subsistema de memoria y la persistencia del banco de registros antes de incorporar operaciones aritméticas complejas.

Escenario de prueba

1. **addi $t0, $zero, 42**
   cargar el valor 42 en $t0
2. **sw $t0, 0($zero)**
   escribir el valor en RAM
3. **lw $t1, 0($zero)**
   recuperar el valor en $t1

Recorrido de datos



Resultados esperados

El PC avanza 0x0000 → 0x0004 → 0x0008

$zero permanece protegido mientras $t0 y $t1 aceptan escrituras

La celda 0x00000000 de RAM muestra 42

PC	RAM
0x0000 → 0x0004 → 0x0008	0x00000000 = 42

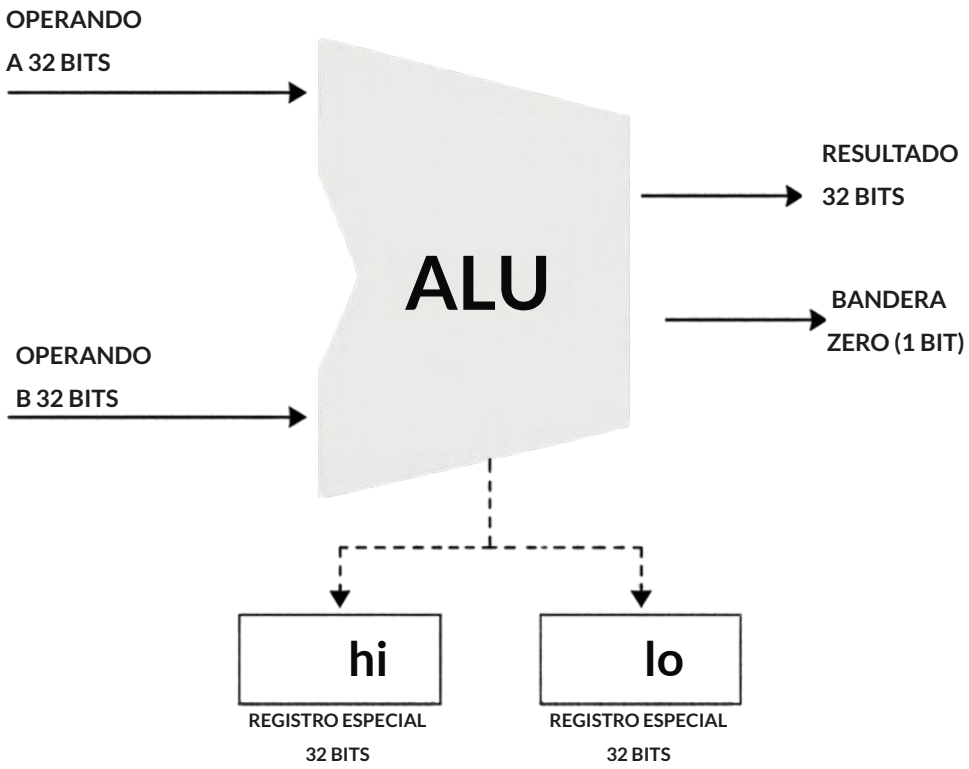
Paso 2. ALU

ALU (Unidad Aritmético Lógica)

El cerebro matemático del procesador.
Ejecuta operaciones aritméticas y lógicas en la etapa **EX (Execute)** .

Recibe dos operandos de 32 bits y una señal ALU Control de 4 bits.
Entrega un resultado de 32 bits

La bandera Zero habilita saltos condicionales (beq) cuando la operación da 0.
Implementada en **ALU java** con switch para multiplexar operaciones.
Usa registros especiales **hi y lo** para resultados de 64 bits.



Procesador a emular

MIPS monociclo de 32 bits

Alcance de la emulación

Procesador MIPS monociclo puro de 32 bits.

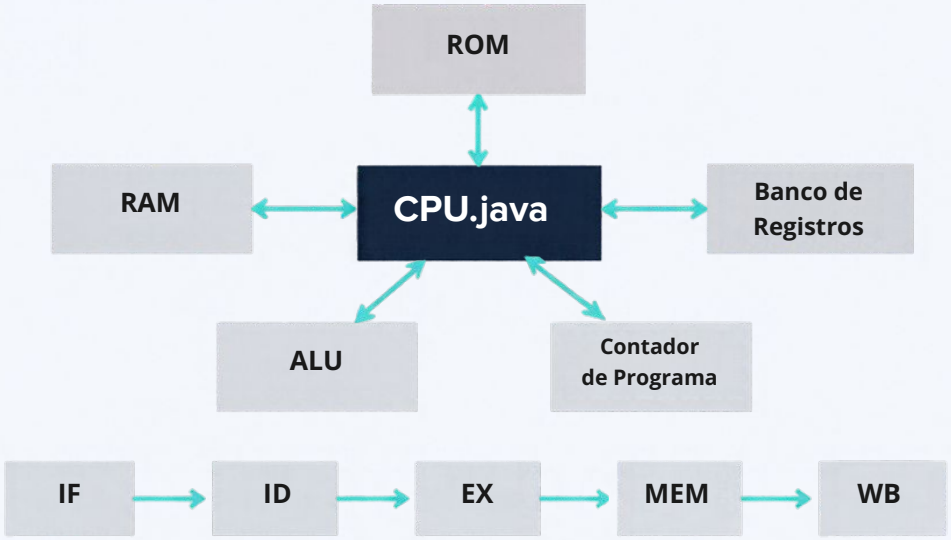
Cada instrucción completa IF, ID, EX, MEM y WB en un solo tick o ciclo de reloj.

Subconjunto de instrucciones

Tipo R: operaciones entre registros: add, sub, slt.

Tipo I: constantes, memoria y condiciones: addi, lw, sw, beq.

Tipo J: salto incondicional que modifica el PC

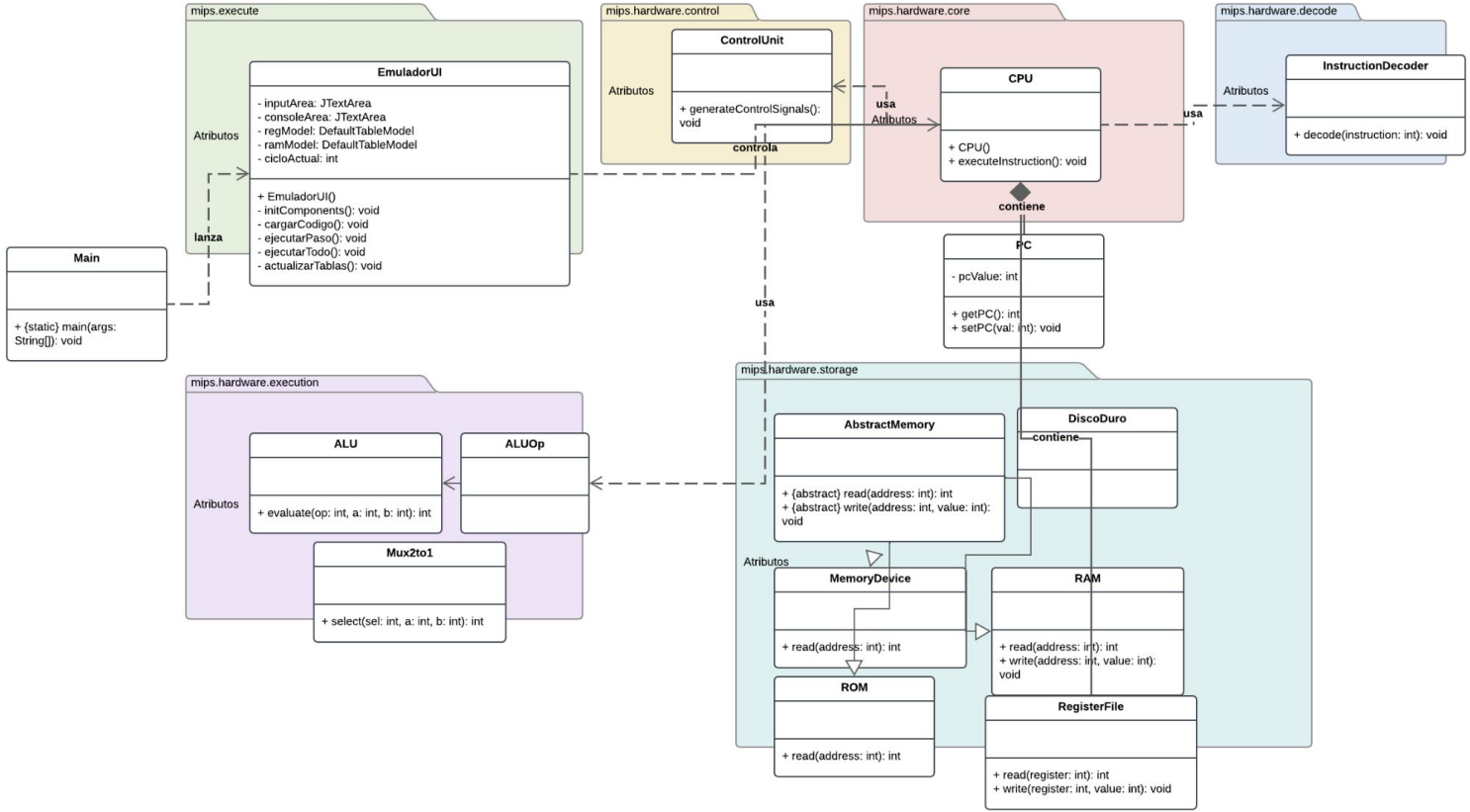


CPU.java

Clase principal que instancia y conecta mediante composición en Java la RAM, la ROM, los Registros, la ALU y el Contador de Programa.

```
public class CPU {
    private final RAM ram;
    private final ROM rom;
    private final RegisterFile registros;
    private final ALU alu;
    private final ProgramCounter pc;
}
```

Diagrama de clases final del proyecto



6 paquetes

12 clases agrupadas

+ 1 clase principal de entrada (Main).

¿Qué es el ISA (Instruction Set Architecture)?

Definición:

- El ISA (*Instruction Set Architecture* o Arquitectura del Conjunto de Instrucciones) es el modelo abstracto de una computadora que define la interfaz entre el software (programas/código máquina) y el hardware (el procesador).

Componentes principales que define:

- **Conjunto de instrucciones:** Las operaciones que el procesador puede ejecutar (aritméticas, lógicas, saltos, transferencias de memoria).
- **Registros:** El almacenamiento interno de alta velocidad disponible para el CPU (como los 32 registros de propósito general en MIPS).
- **Modos de direccionamiento:** Cómo se accede a los datos en la memoria RAM o en las instrucciones.

Importancia:

- Funciona como el "contrato" o puente de lenguaje: le indica al programador y al compilador qué puede hacer físicamente el microprocesador y cómo debe estructurar las instrucciones en binario para que la Unidad de Control las interprete correctamente.

Paso 3. Pruebas técnicas al simulador del procesador MIPS



P1. Sumatoria desde un número inicial a uno final

Objetivo:

Demostrar que el procesador soporta saltos condicionales (`beq`), saltos incondicionales (`j`) y bucles incrementales.

Resultado esperado:

El registro `$10` (`$t2`) terminará con el valor **15** (1+2+3+4+5).

P1. Sumatoria desde un número inicial a uno final

```
"00100000000010000000000000000001", // 0. addi $t0, $zero, 1  (Inicial = 1)
"00100000000010010000000000000101", // 1. addi $t1, $zero, 5  (Final = 5)
"00100000000010100000000000000000", // 2. addi $t2, $zero, 0  (Suma = 0)
"00000001001010000101100000101010", // 3. slt $t3, $t1, $t0   (Bucle: Si Final < Actual, $t3=1)
"00100000000011000000000000000001", // 4. addi $t4, $zero, 1  (Auxiliar = 1)
"00010001011011000000000000000011", // 5. beq $t3, $t4, 3     (Si $t3==1, salta a Syscall)
"00000001010010000101000000100000", // 6. add $t2, $t2, $t0   (Suma = Suma + Actual)
"00100001000010000000000000000001", // 7. addi $t0, $t0, 1    (Actual++)
"00001000000000000000000000000011", // 8. j 3                 (Vuelve a la línea 3)
"00000000000000000000000000001100"  // 9. syscall             (Fin del programa)
```

P2. Factorial de un número

Objetivo:

- Calcular el factorial de 5 (5! = 120) estresando la ALU (con la unidad de multiplicación de 64 bits hi/lo) y la Unidad de Control (mediante bucles).

Resultados Esperados:

- **Registro $t0 (Índice 8):** Inicia en 5 y terminará en **0** (condición que rompe el bucle beq).
- **Registro $t1 (Índice 9):** Acumulador que terminará con el valor final de **120**.
- **Registro Especial lo:** Registrará los saltos de la multiplicación (5, 20, 60 y finalmente **120**).
- **RAM (Dirección 0x0000):** Almacenará el resultado **120** al concluir la ejecución.

P2. Factorial de un número

```
// --- 1. Inicialización ---
"00100000000010000000000000000101", //  0: addi $t0, $zero, 5   (N = 5. Contador externo)
"00100000000010010000000000000001", //  1: addi $t1, $zero, 1   (F = 1. Acumulador del Factorial)
"00100000000010100000000000000001", //  2: addi $t2, $zero, 1   (Constante 1 para comparaciones)

// --- 2. Bucle Externo (Controla el descenso de N) ---
"00010001000010100000000000001001", //  3: outer: beq $t0, $t2, 9 (Si N == 1, salta al final)
"00000000000000000101100000100000", //  4: add $t3, $zero, $zero  (Temp = 0. Guardará la nueva multiplicación)
"00000000000010000110000000100000", //  5: add $t4, $zero, $t0    (Contador interno = N)

// --- 3. Bucle Interno (Multiplica haciendo sumas repetitivas) ---
"00000001011010010101100000100000", //  6: inner: add $t3, $t3, $t1 (Temp = Temp + F)
"00100001100011001111111111111111", //  7: addi $t4, $t4, -1        (Contador interno--)
"00010001100000000000000000000001", //  8: beq $t4, $zero, 1        (Si interno == 0, termina multiplicación)
"00001000000000000000000000000110", //  9: j 6                      (Repite bucle interno)

// --- 4. Actualización de variables ---
"00000000000010110100100000100000", // 10: update: add $t1, $zero, $t3 (F = Temp. Actualiza el factorial)
"00100001000010001111111111111111", // 11: addi $t0, $t0, -1        (N--)
"00001000000000000000000000000011", // 12: j 3                      (Repite bucle externo)

// --- 5. Fin del programa y Escritura ---
"10101100000010010000000000000000", // 13: end: sw $t1, 0($zero)    (Inyecta el 120 en RAM[0])
"00000000000000000000000000001100"  // 14: syscall                  (Detiene el reloj del procesador)
```

P3. Integral definida desde un punto inicial a uno final

Integral definida [2, 4] de f(x)=x

Objetivo:

Demostrar operaciones complejas anidadas (cálculo de áreas) usando la ALU a su máxima capacidad. Calcula: `(4^2 / 2) - (2^2 / 2)`.

Resultado esperado:

El registro `$15` (`$t7`) terminará con el valor **6**.

P3. Integral definida desde un punto inicial a uno final

```
"00100000000010000000000000000010", // 0. addi $t0, $zero, 2  (Límite inferior)
"00100000000010010000000000000100", // 1. addi $t1, $zero, 4  (Límite superior)
"00100000000010100000000000000000", // 2. addi $t2, $zero, 0  (Acumulador = 0)
"00000000000010010110000000100000", // 3. add $t4, $zero, $t1 (Copia del superior para cuadrado)
"00000001010010010101000000100000", // 4. add $t2, $t2, $t1   (Loop cuadrado superior)
"00100001100011001111111111111111", // 5. addi $t4, $t4, -1
"00010001100000000000000000000001", // 6. beq $t4, $zero, 1   (Rompe loop si termina)
"00001000000000000000000000000100", // 7. j 4
"00100000000010110000000000000000", // 8. addi $t3, $zero, 0  (Acumulador = 0)
"00000000000010000110100000100000", // 9. add $t5, $zero, $t0 (Copia inferior para cuadrado)
"00000001011010000101100000100000", // 10. add $t3, $t3, $t0  (Loop cuadrado inferior)
"00100001101011011111111111111111", // 11. addi $t5, $t5, -1
"00010001101000000000000000000001", // 12. beq $t5, $zero, 1  (Rompe loop si termina)
"00001000000000000000000000001010", // 13. j 10
"00000001010010110111000000100010", // 14. sub $t6, $t2, $t3  (Resta cuadrados: 16 - 4 = 12)
"00100000000011110000000000000000", // 15. addi $t7, $zero, 0 (Resultado Final)
"00100000000110000000000000000010", // 16. addi $t8, $zero, 2 (Divisor = 2)
"00000001110110001100100000101010", // 17. slt $t9, $t6, $t8  (Loop de división 12 / 2)
"00100000000000010000000000000001", // 18. addi $at, $zero, 1
"00010011001000010000000000000011", // 19. beq $t9, $at, 3    (Fin de la división)
"00000001110110000111000000100010", // 20. sub $t6, $t6, $t8
"00100001111011110000000000000001", // 21. addi $t7, $t7, 1
"00001000000000000000000000010001", // 22. j 17
"00000000000000000000000000001100"  // 23. syscall
};
```

P4. Promedio de un vector fijo de elementos

Objetivo: Guardar los 3 números físicamente en la Memoria RAM (`sw`), y luego recuperarlos uno por uno desde la RAM hacia los registros (`lw`) para sumarlos y promediarlos.

Resultado esperado:

- La RAM (esquina inferior derecha) mostrará los números 15, 20 y 25
- El registro `$13` (`$t5`) terminará con el promedio exacto: **20**

P4. Promedio de un vector fijo de elementos

```
"00100000000010000000000000001111", //  0: addi $t0, $zero, 15 (Valor 1 = 15)
"00100000000010010000000000010100", //  1: addi $t1, $zero, 20 (Valor 2 = 20)
"00100000000010100000000000011001", //  2: addi $t2, $zero, 25 (Valor 3 = 25)
"00100000000010110000000000000000", //  3: addi $t3, $zero, 0  (Puntero RAM = 0)
"10101101011010000000000000000000", //  4: sw $t0, 0($t3)      (Guarda 15 en RAM[0])
"10101101011010010000000000000100", //  5: sw $t1, 4($t3)      (Guarda 20 en RAM[4])
"10101101011010100000000000001000", //  6: sw $t2, 8($t3)      (Guarda 25 en RAM[8])

// --- 2. Lectura desde RAM (Load) y Sumatoria ---
"10001101011011000000000000000000", //  7: lw $t4, 0($t3)      (Carga 15 desde RAM)
"10001101011011010000000000000100", //  8: lw $t5, 4($t3)      (Carga 20 desde RAM)
"10001101011011100000000000001000", //  9: lw $t6, 8($t3)      (Carga 25 desde RAM)
"00000001100011010110000000100000", // 10: add $t4, $t4, $t5   (Suma parcial: 15 + 20)
"00000001100011100110000000100000", // 11: add $t4, $t4, $t6   (Suma total = 60 en $t4)

// --- 3. División por restas sucesivas (Promedio) ---
"00100000000011010000000000000000", // 12: addi $t5, $zero, 0  ($13/$t5 = Promedio. Inicia en 0)
"00100000000011100000000000000011", // 13: addi $t6, $zero, 3  ($t6 = Divisor = 3)
"00100000000000010000000000000001", // 14: addi $at, $zero, 1  (Constante 1 para el beq)
"00000001100011100111100000101010", // 15: loop: slt $t7, $t4, $t6 (¿Suma < 3?)
"00010001111000010000000000000011", // 16: beq $t7, $at, 3     (Si es menor, salta a syscall)
"00000001100011100110000000100010", // 17: sub $t4, $t4, $t6   (Suma = Suma - 3)
"00100001101011010000000000000001", // 18: addi $t5, $t5, 1    (Promedio++)
"00001000000000000000000000001111", // 19: j 15                (Regresa al inicio del loop)
"00000000000000000000000000001100"  // 20: end: syscall        (Fin. Promedio 20 queda en $t5/$1;
```

P5. Cálculo de un perceptrón con entradas fijas

Objetivo:

Demostrar una red neuronal muy básica. Entradas y Pesos fijos: `(Entrada1 * Peso1) + (Entrada2 * Peso2) + Bias`. *Formula:* `(3) + (2) + (-4) = 1`. Si la función de activación (suma) es mayor a 0, la neurona dispara un 1; si no, un 0.

Resultado esperado:

El registro `$12` (`$t4`) será **1** (La neurona se activó).

P5. Cálculo de un perceptrón con entradas fijas

```
"00100000000010000000000000000011", // 0. addi $t0, $zero, 3  (Entrada1 * Peso1 = 3)
"00100000000010010000000000000010", // 1. addi $t1, $zero, 2  (Entrada2 * Peso2 = 2)
"00100000000010101111111111111100", // 2. addi $t2, $zero, -4 (Bias = -4)
"00000001000010010101100000100000", // 3. add $t3, $t0, $t1   (Suma 1)
"00000001011010100101100000100000", // 4. add $t3, $t3, $t2   (Suma total: 3 + 2 - 4 = 1)
"00100000000011000000000000000000", // 5. addi $t4, $zero, 0  (Output por defecto = 0)
"00000000000010110111000000101010", // 6. slt $t6, $zero, $t3 (¿Es Suma > 0?)
"00010001110000000000000000000001", // 7. beq $t6, $zero, 1   (Si no es mayor a 0, salta
activación)
"00100000000011000000000000000001", // 8. addi $t4, $zero, 1  (¡Neurona activada! Output = 1)
"00000000000000000000000000001100"  // 9. syscall            (Fin)
```

P6. Serie de Fibonacci

```
"00100000000010000000000000001000", //  0: addi $t0, $zero, 8 (Iteraciones)
"00100000000010010000000000000000", //  1: addi $t1, $zero, 0 (F0 = 0)
"00100000000010100000000000000001", //  2: addi $t2, $zero, 1 (F1 = 1)
"00100000000011000000000000000000", //  3: addi $t4, $zero, 0 (Puntero = 0)
"10101101100010010000000000000000", //  4: sw $t1, 0($t4)     (Guarda F0)
"00100001100011000000000000000100", //  5: addi $t4, $t4, 4   (Puntero + 4)
"10101101100010100000000000000000", //  6: sw $t2, 0($t4)     (Guarda F1)
"00100001100011000000000000000100", //  7: addi $t4, $t4, 4   (Puntero + 4)
"00010001000000000000000000000111", //  8: loop: beq $t0, $zero, end
"00000001001010100101100000100000", //  9: add $t3, $t1, $t2  (F_n = F_n-1 + F_n-2)
"10101101100010110000000000000000", // 10: sw $t3, 0($t4)     (Guarda resultado)
"00000000000010100100100000100000", // 11: add $t1, $zero, $t2
"00000000000010110101000000100000", // 12: add $t2, $zero, $t3
"00100001100011000000000000000100", // 13: addi $t4, $t4, 4   (Avanza puntero)
"00100001000010001111111111111111", // 14: addi $t0, $t0, -1  (Decrementa loop)
"00001000000000000000000000001000"  // 15: j loop             (Regresa a instr 8)
```

Reflexiones finales

Conclusiones

El desarrollo e implementación del emulador del procesador MIPS de 32 bits en lenguaje Java permitió cumplir de manera exitosa con el objetivo fundamental de replicar fielmente el comportamiento físico del hardware y su ciclo de cómputo a bajo nivel, sin recurrir a abstracciones innecesarias. A lo largo del proyecto, la integración de los distintos componentes demostró los siguientes logros clave:

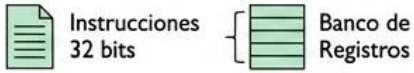
1. **Arquitectura y Filosofía RISC:** Se logró estructurar de forma precisa el diseño de conjunto de instrucciones reducido (RISC), implementando instrucciones de tamaño fijo de 32 bits y el modelo de almacenamiento *Load/Store*, donde las operaciones de cálculo se ejecutan exclusivamente dentro del banco de registros de propósito general y el acceso a memoria se gestiona mediante instrucciones específicas (lw y sw). Asimismo, la protección del registro $zero (cableado físicamente a cero) se garantizó correctamente a nivel de software.
2. **Modelo Harvard y Subsistema de Memoria:** La separación física y lógica entre la memoria de instrucciones (ROM.java) y la memoria de datos (RAM.java) emuló con éxito el modelo Harvard. Esto permitió un acceso independiente y sin colisiones en un mismo ciclo de ejecución, validado mediante pruebas técnicas de persistencia y recorrido de datos entre los registros y la memoria RAM.
3. **Ciclo de Instrucción y Decodificación (ISA):** Mediante la estructuración modular del proyecto en 6 paquetes y 12 clases (destacando la Unidad de Control, la ALU, el Contador de Programa PC y el InstructionDecoder), se simuló de manera exacta el ciclo de instrucción en sus cinco etapas estrictas: Búsqueda (IF), Decodificación (ID), Ejecución (EX), Acceso a Memoria (MEM) y Escritura (WB). La correcta interpretación de los formatos de instrucción (Tipos R, I y J) facilitó la ejecución de programas complejos en pruebas de usuario final, tales como sumatorias, factoriales, integrales definidas, cálculos de promedios en vectores y redes neuronales básicas tipo perceptrón.

En conclusión, este proyecto consolidó los fundamentos teóricos y prácticos de la arquitectura de computadoras, demostrando que Java es un lenguaje idóneo para modelar con precisión circuitería digital, lógica de control y sistemas basados en microprocesadores.

Conclusiones

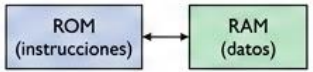
El desarrollo e implementación del emulador del procesador MIPS de 32 bits en lenguaje Java permitió cumplir de manera exitosa con el objetivo fundamental de replicar fielmente el comportamiento físico del hardware y su ciclo de cómputo a bajo nivel, sin recurrir a abstracciones innecesarias. A lo largo del proyecto, la integración de los distintos componentes demostró los siguientes logros clave:

1. Arquitectura y Filosofía RISC



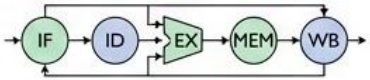
Se logró estructurar de forma precisa el diseño de conjunto de instrucciones reducido (RISC), implementando instrucciones de tamaño fijo de 32 bits y el modelo de almacenamiento Load/Store, donde las operaciones de cálculo se ejecutan exclusivamente dentro del banco de registros de propósito general y el acceso a memoria se gestiona mediante instrucciones específicas (lw y sw). Asimismo, la protección del registro $zero (cableado físicamente a cero) se garantizó correctamente a nivel de software.

2. Modelo Harvard y Subsistema de Memoria



La separación física y lógica entre la memoria de instrucciones (ROM.java) y la memoria de datos (RAM.java) emuló con éxito el modelo Harvard. Esto permitió un acceso independiente y sin colisiones en un mismo ciclo de ejecución, validado mediante pruebas técnicas de persistencia y recorrido de datos entre los registros y la memoria RAM.

3. Ciclo de Instrucción y Decodificación (ISA)



Mediante la estructuración modular del proyecto en 6 paquetes y 12 clases (destacando la Unidad de Control, la ALU, el Contador de Programa PC y el InstructionDec), se simuló de manera exacta el ciclo de instrucción en sus cinco etapas estrictas: as Búsqueda (IF), Decodificación (ID), Ejecución (EX), Acceso a Memoria (MEM) y Escritura (WB). La correcta interpretación de los formatos de instrucción (Tipos R, I y J) facilitó la ejecución de programas complejos en pruebas de usuario final, tales como sumatorias, factoriales, integrales definidas, cálculos de promedios en vectores y redes neuronales básicas tipo perceptrón.

En conclusión, este proyecto consolidó los fundamentos teóricos y prácticos de la arquitectura de computadoras, demostrando que Java es un lenguaje idóneo para modelar con precisión circuitería digital, lógica de control y sistemas basados en microprocesadores.

Trabajo a Futuro

Diseño de Interfaz

- Renovación de paleta de colores.
- Optimización de botones y transiciones.
- Mejoras integrales en la consola.

Diagramación Automática

Representación visual y paso a paso de cada fase del ciclo de cómputo para mayor claridad técnica.

Disponibilidad Abierta

Generación de un repositorio en GitHub para garantizar acceso y disponibilidad al público en general.



Universidad Veracruzana

Región Poza Rica - Tuxpan

Facultad de Ingeniería en Electrónica y Comunicaciones

Ingeniería en Tecnologías Computacionales

Emulación del Procesador [MIPS] en lenguaje [Java]

Experiencia educativa: Microprocesadores y Microcontroladores

Presenta (n):

S24021393 Valdez Avila Moises Alejandro

S24006912 Velazquez Garcia Alexis David

S24006869 Del Angel Garcia Ian Yazid

S24006890 Perez Nuñez Genaro

10 de Octubre de 2026

"Lis de Veracruz: Arte, Ciencia, Luz"